

# Pretty-print SSH and PGP public keys with details

**Author:** Marcos de Carvalho

**Date:** 2026-05-03

Reads SSH public key files and a GnuPG public keyring, then prints paths, SSH key text, PGP identities, long key IDs, fingerprints, and armored public key blocks in a formatted report.

## Language

bash

## Category

cryptography

## Command

```
bash -c 'sshdir=${1:-$HOME/.ssh}; gpghome=${2:-$HOME/.gnupg}; printf "\033[1m=== SSH PUBLIC KEYS (%s) ===\033[0m\n" "$sshdir"; mapfile -d "" s < <(find "$sshdir" -type f \( -name "*.pub" -o -name authorized_keys \) -print0 2>/dev/null); ((${#s[@]})) || echo "No SSH public key files found."; for f in "${s[@]}"; do printf "\n\033[1;34m%s\033[0m\n" "$f"; ssh-keygen -lf "$f" 2>/dev/null | sed "s/^/ details: /"; sed "s/^/ key: /" "$f"; done; printf "\n\033[1m=== PGP PUBLIC KEYS (%s) ===\033[0m\n" "$gpghome"; if [ -e "$gpghome/pubring.kbx" ] || [ -e "$gpghome/pubring.gpg" ]; then GNUPGHOME="$gpghome" gpg --batch --list-public-keys --with-fingerprint --with-subkey-fingerprint --keyid-format long 2>/dev/null | sed "s/^/ /"; printf "\n\033[1m=== ARMORED PGP PUBLIC KEY BLOCKS ===\033[0m\n"; GNUPGHOME="$gpghome" gpg --batch --armor --export 2>/dev/null; else echo "No GnuPG public keyring found."; fi' _
```

## Explanation

The command uses a `bash -c` wrapper so both optional paths are supplied as trailing arguments. `SSH_DIR` defaults to `~/ssh` and is searched recursively for `*.pub` files and `authorized_keys` files. Each matching SSH file is labeled, summarized with `ssh-keygen -lf`, and printed with indentation. `GNUPG_HOME` defaults to `~/gnupg`; when it contains `pubring.kbx` or `pubring.gpg`, GnuPG lists

public keys with long key IDs, primary fingerprints, and subkey fingerprints, then exports all public keys as ASCII-armored blocks. The `_` placeholder acts as `$0` inside `bash -c` so `$1` and `$2` map cleanly to `SSH_DIR` and `GNUPG_HOME`.

## Tags

ssh, pgp, gpg, public-keys, fingerprints, cryptography, audit, pretty-print

## Dependencies

bash, find, ssh-keygen, sed, gpg

## Arguments

1. **SSH\_DIR** (Optional): Directory to search recursively for SSH public key files. Defaults to `~/ssh`.  
Default: `~/ssh`
2. **GNUPG\_HOME** (Optional): GnuPG home directory whose public keyring will be listed and exported. Defaults to `~/gnupg`.  
Default: `~/gnupg`

## Examples

1. 

```
bash -c 'sshdir=${1:-$HOME/.ssh}; gpghome=${2:-$HOME/.gnupg}; printf
"\{\}033[1m=== SSH PUBLIC KEYS (%s) ===\{\}033[0m\{\}n" "$sshdir";
mapfile -d "" s < <(find "$sshdir" -type f \{\} ( -name "*.pub" -o
-name authorized_keys \{\}) -print0 2>/dev/null); ((${#s[@]}) ||
echo "No SSH public key files found."; for f in "${s[@]}"; do printf
"\{\}n\{\}033[1;34m%s\{\}033[0m\{\}n" "$f"; ssh-keygen -lf "$f" 2>/dev/null
| sed "s/^/ details: /"; sed "s/^/ key: /" "$f"; done; printf
"\{\}n\{\}033[1m=== PGP PUBLIC KEYS (%s) ===\{\}033[0m\{\}n" "$gpghome";
if [ -e "$gpghome/pubring.kbx" ] || [ -e "$gpghome/pubring.gpg"
]; then GNUPGHOME="$gpghome" gpg --batch --list-public-keys --with-fingerprint
--with-subkey-fingerprint --keyid-format long 2>/dev/null | sed
"s/^/ /"; printf "\{\}n\{\}033[1m=== ARMORED PGP PUBLIC KEY BLOCKS
===\{\}033[0m\{\}n"; GNUPGHOME="$gpghome" gpg --batch --armor --export
2>/dev/null; else echo "No GnuPG public keyring found."; fi' _
- Print SSH public keys from ~/ssh and PGP public keys from ~/gnupg
```
2. 

```
bash -c 'sshdir=${1:-$HOME/.ssh}; gpghome=${2:-$HOME/.gnupg}; printf
"\{\}033[1m=== SSH PUBLIC KEYS (%s) ===\{\}033[0m\{\}n" "$sshdir";
mapfile -d "" s < <(find "$sshdir" -type f \{\} ( -name "*.pub" -o
-name authorized_keys \{\}) -print0 2>/dev/null); ((${#s[@]}) ||
```

```

echo "No SSH public key files found."; for f in "${s[@]}"; do printf
"\{\}n\{\}033[1;34m%s\{\}033[0m\{\}n" "$f"; ssh-keygen -lf "$f" 2>/dev/null
| sed "s/^/ details: /"; sed "s/^/ key: /" "$f"; done; printf
"\{\}n\{\}033[1m=== PGP PUBLIC KEYS (%s) ===\{\}033[0m\{\}n" "$gpghome";
if [ -e "$gpghome/pubring.kbx" ] || [ -e "$gpghome/pubring.gpg"
]; then GNUPGHOME="$gpghome" gpg --batch --list-public-keys --with-fingerprint
--with-subkey-fingerprint --keyid-format long 2>/dev/null | sed
"s/^/ /"; printf "\{\}n\{\}033[1m=== ARMORED PGP PUBLIC KEY BLOCKS
===\{\}033[0m\{\}n"; GNUPGHOME="$gpghome" gpg --batch --armor --export
2>/dev/null; else echo "No GnuPG public keyring found."; fi' _
/etc/ssh ~/.gnupg - Print system SSH host public keys from /etc/ssh
and the current user's GnuPG public keyring

```

## Output

```

\u001b[1m=== SSH PUBLIC KEYS (/home/user/.ssh) ===\u001b[0m

\u001b[1;34m/home/user/.ssh/id_ed25519.pub\u001b[0m
details: 256 SHA256:examplefingerprint user@example (
ED25519)
key: ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKey
user@example

\u001b[1m=== PGP PUBLIC KEYS (/home/user/.gnupg) ===\u001b[0m
m
pub      ed25519/0123456789ABCDEF 2026-05-03 [SC]
          0123 4567 89AB CDEF 0123 4567 0123 4567 89AB CDEF
uid              [ultimate] User Example <user@example>
sub      cv25519/FEDCBA9876543210 2026-05-03 [E]
          FEDC BA98 7654 3210 FEDC BA98 7654 3210 FEDC BA98

\u001b[1m=== ARMORED PGP PUBLIC KEY BLOCKS ===\u001b[0m
-----BEGIN PGP PUBLIC KEY BLOCK-----
...
-----END PGP PUBLIC KEY BLOCK-----

```

## Notes

- The SSH section includes \*.pub files and authorized\_keys files under SSH\_DIR.
- ssh-keygen -lf prints one fingerprint line per public key when a file contains multiple keys.
- The PGP section uses the selected GnuPG home directory and requires pubring.kbx or pubring.gpg to exist.

- Use a pager such as `less -R` when the exported PGP public key blocks are long.
- This command is intended for Bash because it uses mapfile and process substitution.

## Warnings

- Public keys are not secret, but they often include names, email addresses, hostnames, or comments that may be sensitive in shared logs or screen-shots.
- The command does not print private SSH or PGP key material.
- GnuPG may create or refresh local keyring metadata such as `trustdb.gpg` when inspecting a GnuPG home that lacks it.

## See Also

- `bulk-gpg-detach-sign-files`

## Status

reviewed

## Safety

caution

## Shell

bash

## Platforms

linux, gnu-linux

## Created At

2026-05-03

**Updated At**

2026-05-03